

Session 5

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

1



SERIAL PERIPHERAL INTERFACE (SPI)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

2

SPI MAIN CHARACTERISTICS.

- Serial synchronous transmission
 - Clock is generated by Controller ("master") - SCLK
- Target ("slave") devices are selected by dedicated Slave Select (SS) Input
 - One line for each target
 - As opposed to sending address packet in I2C
- In general, it allows higher transmission speeds than I2C
 - For short distances, also
- Two data lines
 - Master Out Slave In (MOSI)
 - Master In Slave Out (MISO)
 - Allows full duplex

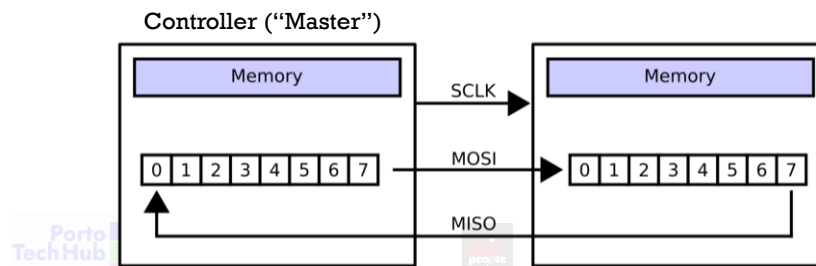
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

3

3

SPI TRANSMISSION

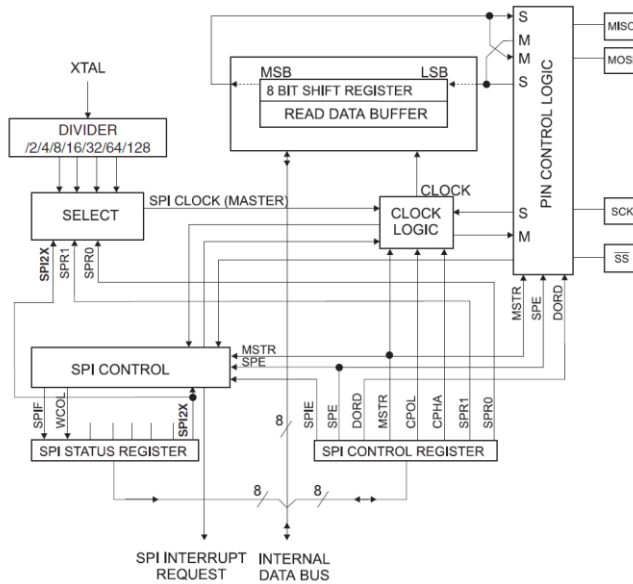
- At each SCLK cycle:
 - A bit is transferred from the Controller to the Target
 - A bit is transferred from Target to the Controller



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

4

4



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

SPI ON ATMEGA128

Registers:

- SPI Control Register (SPCR)
- SPI Status Register (SPSR)
- SPI Data Register (SPDR)

/SS (PB0) in Controller Mode

- As output, used to activate target(s)
- As input, allows another device to take the role of controller
 - SPI is automatically switched to target mode
 - If interrupts are enabled, an interrupt is generated

tander

5

SPCR

Bit	7	6	5	4	3	2	1	0	
	SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0	SPCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

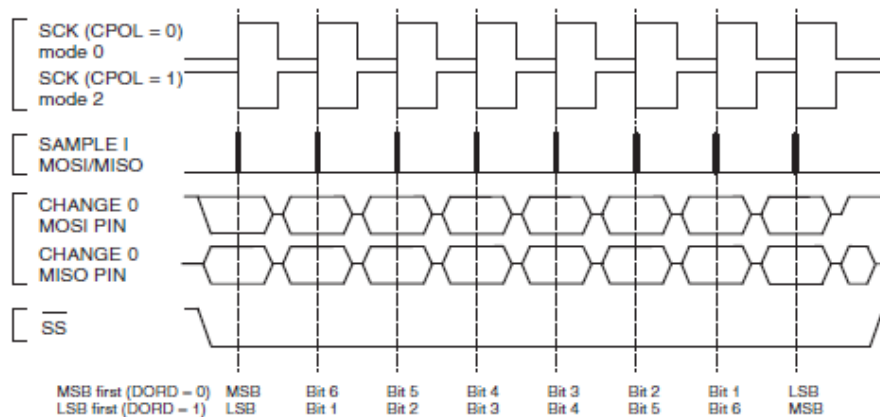
- SPIE:** SPI Interrupt Enable
- SPE:** SPI Enable
- DORD:** Data Order – if set to 1, the LSB is transmitted first
- MSTR:** Master/Slave Select – Controller mode, if set to 1
- SPR1, SPR0:** SPI Clock Rate Select 1 and 0

SPI2X	SPR1	SPR0	SCK Frequency
0	0	0	$f_{osc}/4$
0	0	1	$f_{osc}/16$
0	1	0	$f_{osc}/64$
0	1	1	$f_{osc}/128$
1	0	0	$f_{osc}/2$
1	0	1	$f_{osc}/8$
1	1	0	$f_{osc}/32$
1	1	1	$f_{osc}/64$

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

6

CLOCK POLARITY (CPOL) AND PHASE (CPHA)

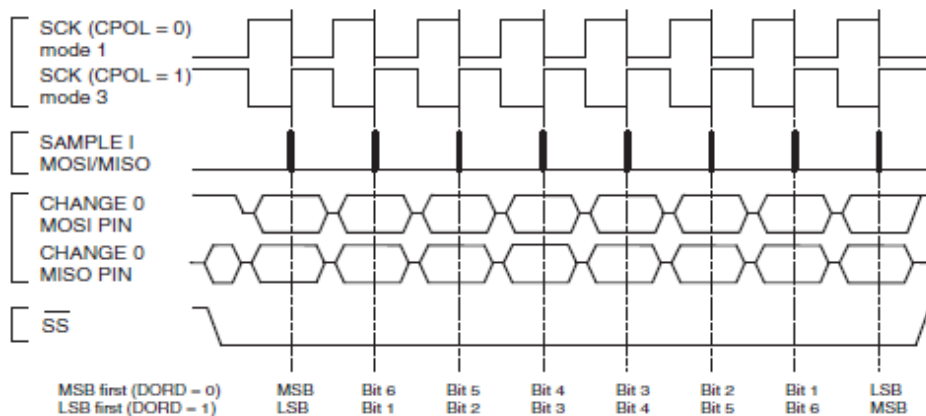


PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

7

7

CLOCK POLARITY (CPOL) AND PHASE (CPHA)



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

8

8

SPSR

Bit	7	6	5	4	3	2	1	0	
	SPIF	WCOL	–	–	–	–	–	SPI2X	SPSR
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **SPIF: SPI Interrupt Flag**
 - Set when:
 - A transfer is complete
 - When /SS is driven low, if /SS configured for input and SPI in Controller mode (MSTR bit set)
 - Cleared when:
 - Executing the interrupt routine, if interrupts enabled
 - Reading the sSPSR followed by accessing the Data Register (SPDR)
- **WCOL: Write COLLision flag** - set if SPDR is written during a data transfer
 - Cleared by reading SPSR followed by accessing SPDR
- **SPI2X: Double SPI Speed Bit** – used to define SCK frequency in master mode (see also SPCR)

techHub isep Instituto Superior de Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



9

SPI AS A CONTROLLER: CODE EXAMPLE

```
void SPI_MasterInit(void) {
    /* Set MOSI and SCK output, all others input */
    DDR_SPI = (1<<DD_MOSI)|(1<<DD_SCK);
    /* Enable SPI, Master, set clock rate fck/16 */
    SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR0);
}

void SPI_MasterTransmit(char cData)
{
    /* Start transmission */
    SPDR = cData;
    /* Wait for transmission complete */
    while(!(SPSR & (1<<SPIF)));
}
```

techHub isep Instituto Superior de Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



10

SPI AS A TARGET: CODE EXAMPLE

```
void SPI_SlaveInit(void) {
    /* Set MISO output, all others input */
    DDR_SPI = (1<<DD_MISO);
    /* Enable SPI */
    SPCR = (1<<SPE);
}
```

```
char SPI_SlaveReceive(void){
    /* Wait for reception complete */
    while(!(SPSR & (1<<SPIF)));
    /* Return data register */
    return SPDR;
}
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

11

11

12 ARDUINO

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

12

ARDUINO

- **Hardware + Software Solution (library)**
 - IDE also available (Arduino IDE 2.x, since 2021)
 - Other IDEs/toolchains can also be used
 - On the other hand, other platforms are also supported (ESP32, STM32, etc.)
 - Most software under GNU General Public Licence (GPL)
 - Hardware is also open-source
- **Several board models available:**
- **Fast prototyping**
- **Also some boards for the professional segment**



Instituto Superior de
Engenharia do Porto



Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

13

13

SOME ARDUINO BOARDS

- **Based on AVR 8-bit microcontrollers:** UNO R3, UNO Mini, UNO WiFi, Nano, Nano Every, Mega 2560, etc.
- **Based on 32-bit ARM microcontrollers**
 - Cortex-M0 (MKR series, Nano 33 IoT, Nano RP2040, Zero)
 - Cortex-M3 (Due)
 - Cortex-M33 with FPU (Nano Matter, Portenta C33)
 - Cortex-M4 with FPU (Nano 33 BLE, UNO R4)
- **Based on Xtensa architecture (ESP32):** Nano ESP32 (contains FPU)
- **With Linux Support:**
 - UNO Q: 64-bit SOC with ARM Cortex-A53 cores (ARM "Application" profile), up to 4 GB RAM
 - Portenta X8 - 4x ARM® Cortex-A53 core up to 1.8GHz
 - Yun Rev2 - Atheros AR9331 (based on MIPS 24K microprocessor@400 MHz)

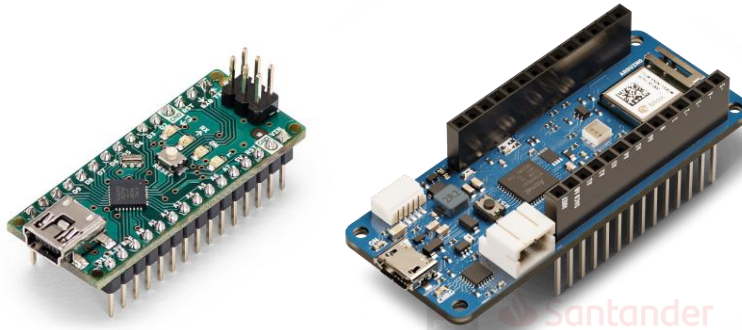
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

14

14

SOME ARDUINO BOARDS

- Nano, MKR



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

15

15

OTHER MANUFACTURERS WITH ARM CORTEX-M SOLUTIONS

- STMicroelectronics (STM32)
- NXP Semiconductors
 - LPC (low power solutions)
 - Kinetis (high-performance solutions)
 - i.MX (including “Application” processors, Linux support, AI support)
- Microchip Technology (SAM)
- Infineon Technologies (XMC line - KIT_XMC4400_DC_V1, for instance)
- Silicon Labs (EFM32ZG Starter Kit, etc.)
- Renesas Electronics

Porto TechHub isep Instituto Superior de Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

16

16

ARDUINO: MAIN.CPP

```
int main(void) {
  init();

  initVariant();

  #if defined(USBCON)
    USBDevice.attach();
  #endif

  setup();

  for (;;) {
    loop();
    if (serialEventRun) serialEventRun();
  }
  return 0;
}
```

- This file belongs to the Arduino library
 - packages\arduino\hardware\avr\1.8.7\cores\main.cpp
- Users just have to define the following function:
 - **setup()**
 - **loop()**
- Call to init() initializes wiring library (wiring.c)
- initVariant() is optional:
 - // Weak empty variant initialization function.
 - // May be redefined by variant files.
 - void initVariant() __attribute__((weak));
 - void initVariant() { }

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

17

17

BASIC ARDUINO EXAMPLE

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);                     // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);                     // wait for a second
}
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

18

18

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}

void digitalWrite(uint8_t pin, uint8_t val)
{
  uint8_t timer = digitalPinToTimer(pin);
  uint8_t bit = digitalPinToBitMask(pin);
  uint8_t port = digitalPinToPort(pin);
  volatile uint8_t *out;

  if (port == NOT_A_PIN) return;

  // If the pin that support PWM output, we need to turn it off
  // before doing a digital write.
  if (timer != NOT_ON_TIMER) turnOffPWM(timer);

  out = portOutputRegister(port);

  uint8_t oldSREG = SREG;
  cli();

  if (val == LOW) {
    *out &= ~bit;
  } else {
    *out |= bit;
  }

  SREG = oldSREG;
}

void pinMode(uint8_t pin, uint8_t mode)
{
  uint8_t bit = digitalPinToBitMask(pin);
  uint8_t port = digitalPinToPort(pin);
  volatile uint8_t *reg, *out;

  if (port == NOT_A_PIN) return;

  // JWS: can I let the optimizer do this?
  reg = portModeRegister(port);
  out = portOutputRegister(port);

  if (mode == INPUT) {
    uint8_t oldSREG = SREG;
    cli();
    *reg &= ~bit;
    *out &= ~bit;
    SREG = oldSREG;
  } else if (mode == INPUT_PULLUP) {
    uint8_t oldSREG = SREG;
    cli();
    *reg &= ~bit;
    *out |= bit;
    SREG = oldSREG;
  } else {
    uint8_t oldSREG = SREG;
    cli();
    *reg |= bit;
    SREG = oldSREG;
  }
}
```



PGSCE-DSISEM - 2025/2026

19

ARDUINO EXAMPLE OF TWI TRANSMISSION

```
#include <Wire.h>
```

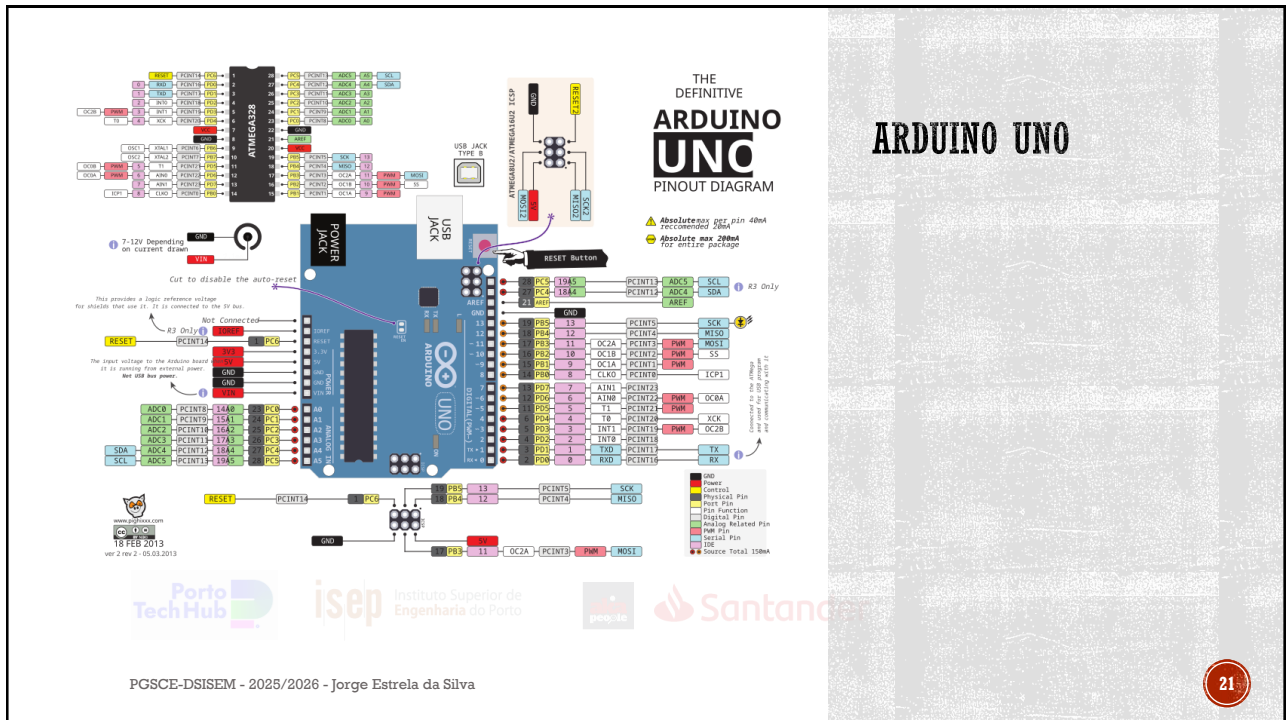
```
void setup() {
  Wire.begin();
  Wire.setClock(100000); //the device joins the bus as a controller
}
```

```
void loop() {
  uint8_t data[] = {2,3};
  Wire.beginTransmission(1); //transmit to target 1 (of 127)
  Wire.write(data, 2);
  Wire.endTransmission(true); //true sends a STOP, false sends a "restart"
}
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

20

20

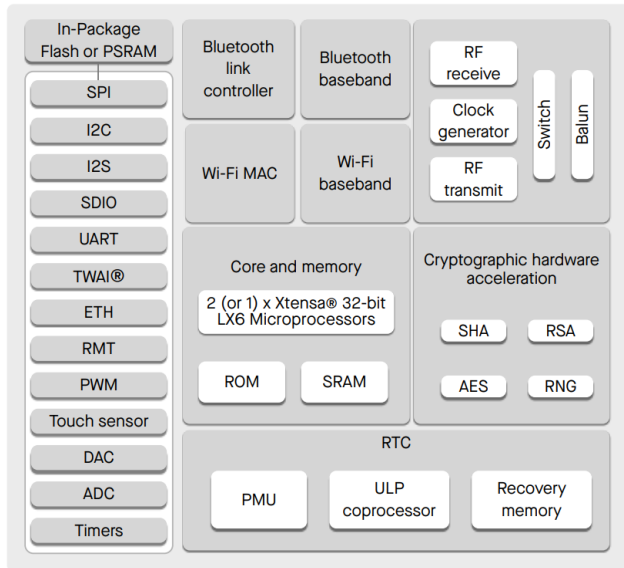


21

ARDUINO UNO (ATMEGA328)

- Some differences with respect to ATmega128
 - No PORTA
 - Timer/Counters with slight differences, more Output Compares, differences on register and bit naming (code must be adapted)
 - Only one USART (USART0)
 - Etc.

22



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

MULTICORE MICROCONTROLLERS: ESP32 (EXAMPLE)

- Other peripherals:
 - I2S - Inter-Integrated Circuit Sound (synchronous serial)
 - SDIO – general I/O (GPS, etc.) on SD interface
 - TWAI - Two-wire Automotive Interface (ESP32 CAN bus implementation)
 - RMT – TX/RX of infrared remote control signals.
 - etc.
- Two “cores”
 - Memory shared between both cores
 - In general, easier to leverage the processing power by using a suitable operating system (FreeRTOS, etc.)

23

Session 6

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva



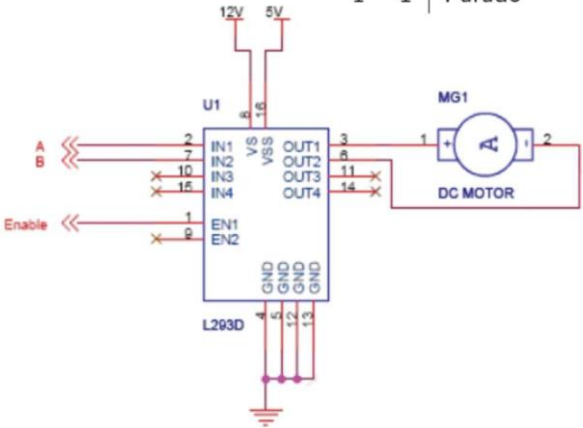
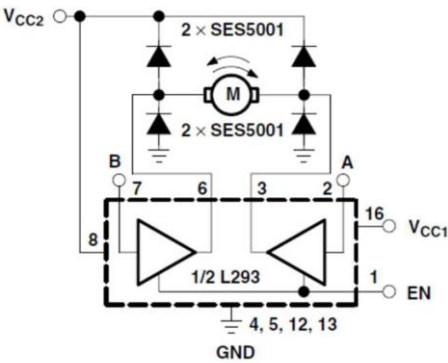
PL EXERCISES

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

25

DC MOTOR DRIVER – L293D

A	B	Descrição
0	0	Parado
0	1	Roda no SPR
1	0	Roda no SCPR
1	1	Parado



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



26

```

void change_dir() {

    if(PORTB & (1 << 5)) {

        PORTB |= (1<<5)|(1<<6); // no potential difference ("voltage") on motor terminals

        _delay_ms(250);          //wait until motor slows down

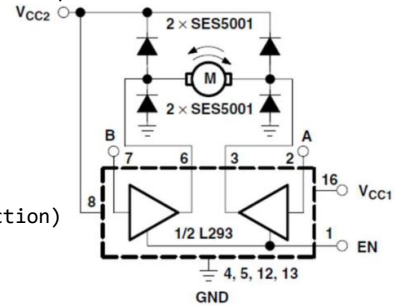
        PORTB &= ~(1 << 5);      //clear PB5 and leave PB6 set
                                //(activate motor on opposite direction)
    } else {

        PORTB |= (1<<5)|(1<<6);

        _delay_ms(250);

        PORTB &= ~(1 << 6);      // clear PB6 and leave PB5 set
                                //(activate motor on opposite direction)
    }
}

```



27

```

#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdio.h>

#define MODE_DIGITAL 1
#define MODE_ANALOG 2

//function prototypes and global variables

uint8_t glob_mode = MODE_DIGITAL;
//...

int main() {
    uint8_t val, old_val = 1;

    init();

    while(1) {
        //check desired direction
        if(val == 0 && old_val == (1<<5))
            glob_change_dir_req = 1;
        old_val = val;

        if(glob_change_dir_req) {
            change_dir();
            glob_change_dir_req = 0;
        }

        if(glob_send_pwm_req) {
            usart_send_byte_as_string(glob_pwm);
            glob_send_pwm_req = 0;
        }

        switch(glob_mode) {
            case MODE_DIGITAL:
                break;
            case MODE_ANALOG:
                glob_pwm = read_adc();
                set_pwm(glob_pwm);
            }
        }
    }
}

```

28

```

ISR(USART1_RX_vect){
    uint8_t stat = UCSR1A;
    uint8_t buf = UDR1;

    //FE - frame error
    if( stat & (1<<FE1))
        return;

    if(glob_mode!=MODE_ANALOG) {
        switch(buf) {
            case 'P':
            case 'p':
                glob_pwm = 0;
                set_pwm(glob_pwm);
                break;
            case '1': //PWM 25%
                glob_pwm = 25;
                set_pwm(glob_pwm);
                break;
            case '2': // PWM 75%
                glob_pwm = 75;
                set_pwm(glob_pwm);
                break;
            case '+':
                if(glob_pwm < 95) {
                    glob_pwm += 5;
                } else
                    glob_pwm = 100;
                set_pwm(glob_pwm);
                break;
            case '-':
                if(glob_pwm > 5) {
                    glob_pwm -= 5;
                } else
                    glob_pwm = 0;
                set_pwm(glob_pwm);
                break;
        }
    }

    switch(buf) {
        case 'I':
        case 'i': //lengthy operation: just set the
                    //flag and defer it to main loop
                    glob_change_dir_req = 1;
                    break;
        case 'B':
        case 'b':
            glob_send_pwm_req = 1;
            break;
        case 'D':
        case 'd':
            glob_mode = MODE_DIGITAL;
            break;
        case 'A':
        case 'a':
            glob_mode = MODE_ANALOG;
            break;
    }
}

```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

29

29

```

void init() {
    DDRB = 0b11100000; //PB7:5 as outputs
    //driver disabled, no direction set
    PORTB = 0b01111111;

    OCR2 = 0; //duty cycle = 0%
    //Prescaler = F_CPU/F_PWM/510 = 16000000 / 500 / 510 = 62.7
    //(1 << 6) -> PWM, phase correct
    TCCR2 |= (1 << 6) | (1 << CS21) | (1 << CS20) | (1 << COM21);
    // TCCR2 = 0b01101011 -> fast PWM, 1 kHz
}

```

• Bit 5:4 – COM21:0: Compare Match Output Mode

COM21	COM20	Description
0	0	Normal port operation, OC2 disconnected.
0	1	Reserved
1	0	Clear OC2 on compare match when up-counting. Set OC2 on compare match when downcounting.
1	1	Set OC2 on compare match when up-counting. Clear OC2 on compare match when downcounting.

• Bit 2:0 – CS22:0: Clock Select

CS22	CS21	CS20	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clkI/O (No prescaling)
0	1	0	clkI/O/8 (From prescaler)
0	1	1	clkI/O/64 (From prescaler)
1	0	0	clkI/O/256 (From prescaler)
1	0	1	clkI/O/1024 (From prescaler)
1	1	0	External clock source on T2 pin. Clock on falling edge
1	1	1	External clock source on T2 pin. Clock on rising edge

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

30

30

```

void set_pwm(uint8_t pwm) {           //convert from 0..100 to 0..255
    uint16_t tmp = pwm;
    tmp *= 255;
    tmp /= 100;
    OCR2 = tmp;
}

```



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

31

31

```

void change_dir() {
    if(PORTB & (1 << 5)) {

        PORTB |= (1<<5)|(1<<6); //Apply 0 V to motor
        set_pwm(100);

        _delay_ms(250);           //wait until motor slows down
        set_pwm(glob_pwm);        //restore PWM duty cycle (no potential difference on motor terminals yet)
        TIFR |= (1<<TOV2);        //wait until a new PWM cycle begins, so that the new OCR2 value is used
        while( (TIFR & (1<<TOV2))==0);

        PORTB &= ~(1 << 5);       //clear PB5 and leave PB6 set
                                   //(activate motor on opposite direction)

    } else {

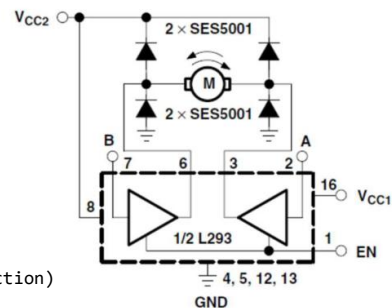
        PORTB |= (1<<5)|(1<<6);
        set_pwm(100);

        _delay_ms(250);
        set_pwm(glob_pwm);
        TIFR |= (1<<TOV2);
        while( (TIFR & (1<<TOV2))==0);

        PORTB &= ~(1 << 6);       // clear PB6 and leave PB5 set
                                   //(activate motor on opposite direction)

    }
}

```



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

32

32

```

void init_usart() {
    cli();

    //9600 bps
    // U2X1 = 1 => baudrate = FOSC/(UBRR+1)/8
    //      <=> UBRR = FOSC/8/baudrate - 1
    UCSR1A |= (1<<U2X1);
    UBRR1H = 0;
    UBRR1L = 207;

    //enable TX and RX
    UCSR1B |= (1<<RXEN1)|(1<<TXEN1);
    //8 bits data word
    UCSR1C |= (1<<UCSZ11)|(1<<UCSZ10);
    //2 stop bits
    UCSR1C |= (1<<USBS1);

    //Enable RX interrupt
    UCSR1B |= (1<<RXCIE1);
}

```

Bit	7	6	5	4	3	2	1	0	
	UMSELn	UPMn1	UPMn0	USBSn	UCSZn1	UCSZn0	UCPOLn		UCSRnC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	1	1	0	

UMSELn	Modo
0	Assíncrono
1	Síncrono

UPMn1	UPMn0	Paridade
0	0	Sem paridade
0	1	(Reservado)
1	0	Paridade par
1	1	Paridade ímpar

USBSn	Stop bits
0	1 stop bit
1	2 stop bits

UCSZn2	UCSZn1	UCSZn0	Comprimento da palavra
0	0	0	5 bits
0	0	1	6 bits
0	1	0	7 bits
0	1	1	8 bits
1	0	0	(Reservado)
1	0	1	(Reservado)
1	1	0	(Reservado)
1	1	1	9 bits

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

33

33

```

//Takes at most ~4 ms
void usart_send_byte_as_string(uint8_t x) {
    char str1[5];
    sprintf(str1, "%u\n", x);

    char *buf = str1;
    while(*buf != 0) {
        while( ( UCSR1A & (1 << UDRE1) ) == 0 ); //busy wait until DR Empty

        UDR1 = *buf;
        ++buf;
    }
}

```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

34

34



RASPBERRY PI

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

35

DEVELOPMENT FOR THE RASPBERRY PI PLATFORM

- You should have a Linux system on your personal computer.
 - You may use a virtual machine.
 - WSL (Windows 10 and 11), Hyper-V (Windows 10 and 11 Pro), VirtualBox, etc.
 - <https://learn.microsoft.com/en-us/windows/wsl/install>
 - <https://www.virtualbox.org/wiki/Downloads>
 - Macs with ARM CPU (most models since 2021): choose a distribution with support for ARM aarch64.
 - For instance, <https://www.fedoraproject.org/workstation/download/>
- Please bring a USB pen drive (1 GB or more) to the PL classes

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



36